

NFC Relay Goes Local: How AI Is Accelerating a New Wave of Independent Malware Developers

 cleafy.com/cleafy-labs/nfc-relay-goes-local-how-ai-is-accelerating-a-new-wave-of-independent-malware-developers

Simone Mattia, Federico Valentini

Key points

- The Cleafy Threat Intelligence team has identified and analyzed two new Android NFC relay malware families: **DevilNFC** and **NFCMultiPay**. While these families are developed by distinct, unrelated actors with no shared code or infrastructure, both are actively used to successfully launch NFC relay attacks targeting **European and LATAM banking customers**.
- These two NFC relay toolkits are being developed and operated outside the Chinese-speaking MaaS ecosystem: DevilNFC carries an exclusively **Spanish-speaking attribution**, while NFCMultiPay's developer fingerprint is **Portuguese (Brazilian)**. Local groups are no longer buying access to Chinese platforms; they are **building their own**.
- Both families show development characteristics consistent with **generative AI tooling**. Open-source LLM models without safety controls and leaked malware codebases on public repositories are collectively **lowering the technical threshold** for building functional malware.
- DevilNFC represents the more advanced of the two families: a **single APK** serving both the victim and the fraudster, operating at the system level via a **hooking framework** to intercept NFC traffic. NFCMultiPay takes a simpler approach, implementing the full relay in **pure Java** with no root requirement and no native code.
- Both families **collect the victim's card PIN** as a core step of the attack flow, not an optional enhancement. DevilNFC further locks the victim inside the malicious interface via **Kiosk Mode**, preventing any escape while the relay completes. The convergence of these capabilities across two independently developed families confirms they have become the new baseline for NFC relay operations.

Executive Summary

Since the start of 2026, Cleafy's Threat Intelligence and Response (TIR) team has identified and analysed two previously undocumented Android malware families, **DevilNFC** and **NFCMultiPay**, both of which are actively conducting NFC relay attacks against European banking customers. The two families were developed independently, by unrelated Threat Actors (TAs), with no shared code or infrastructure. The concurrent appearance across overlapping target geographies represents a significant turning point in the NFC relay threat landscape: a technique previously confined to Chinese-speaking MaaS operations has been **independently replicated by new local groups**, with DevilNFC attributed to Spanish-speaking TAs and NFCMultiPay carrying a Portuguese (Brazilian) developer fingerprint.

DevilNFC employs an asymmetric architecture in which a single APK serves both roles in a relay attack: a passive reader on the victim's device and a system-level card emulator on the attacker's rooted device, achieved via a hooking framework that intercepts NFC traffic below the Android API layer. **NFCMultiPay** demonstrates that the same outcome can be achieved with a simpler approach: pure Java, no root, no native libraries, two Android phones, and a cloud broker. Multiple confirmed builds have been identified, targeting financial institutions across Europe. Beyond the relay mechanism, both families invest significantly in the **social engineering layer**. DevilNFC locks the victim inside a fake banking interface via Kiosk Mode, preventing the device from being left while the relay completes. NFCMultiPay guides the victim through a branded UI that impersonates the target institution, maintaining the deception without device locking. Both families **capture the victim's card PIN** as a baseline operational requirement, not an optional enhancement, extending the fraud surface beyond contactless limits to unconstrained ATM withdrawals and chip-and-PIN transactions at any point of sale globally.

Both exhibit development characteristics consistent with AI-assisted tooling, including over-engineered phishing templates in DevilNFC and LLM-characteristic emoji-formatted logging in NFCMultiPay. This trend is **independently confirmed** by a recent investigation of ESET, which in April 2026 identified a new NGate variant targeting Brazilian users, in which the injected malicious code carries the same AI development indicators and Portuguese (Brazilian) strings.

Introduction

One year ago, **Cleafy** Threat Intelligence published its analysis of SuperCard X, a Chinese-speaking Malware-as-a-Service platform enabling **real-time NFC relay attacks** against EU banking customers. That report documented a structured criminal ecosystem built around NFC relay fraud: a MaaS operator providing ready-made tooling to affiliates, a two-application

Reader/Tapper architecture, and a social engineering layer designed to extract card data without the physical card ever leaving the victim's possession.

While SuperCard X did not originate this threat class, it was responsible for its **industrialization**. The underlying technique traces back to NFCGate, an open-source research tool developed by the Technical University of Darmstadt. Most of the NFC relay families documented through 2025 share the same architectural DNA: NFCGate's relay core, repackaged in a new wrapper application and distributed under a new name. The TAs were **not innovating on the technique**; they were iterating on the delivery, branding, and operational model, built on a foundation that had not fundamentally changed since its academic publication.

Throughout 2025, multiple mobile security vendors reported a sharp **increase in NFC relay campaigns** targeting European banking customers. The campaigns differed in branding, target geography, and affiliate operator, but converged on a single common denominator: **Chinese-speaking developers** controlling the **tooling layer**. Most of the documented families traced back to the same NFCGate foundation, distributed through a similar MaaS channel model, promoted through the same Telegram ecosystem, selling access to operators who rarely had the technical background to build the tools themselves. The barrier to entry was not insurmountable, but high enough to keep the threat concentrated among a small number of technically sophisticated groups: **Chinese-speaking actors** had effectively established a **monopoly** over the development of **NFC relay malware**, but in 2026, that monopoly ended, thanks to the convergence of open-source tooling and generative AI.

The Rising of Local NFC Relay Groups

In early 2026, Cleafy Threat Intelligence identified two Android NFC relay families that, despite sharing no code or infrastructure, tell a similar story: **local TAs** in EU and LATAM are **building their own NFC relay toolkits** and deploying them against their local banking customers.

The two families sit at opposite ends of the technical spectrum. **DevilNFC** is more sophisticated: a **single APK** that operates as a passive reader on the victim's unrooted device and transforms into a system-level card emulator on the attacker's rooted hardware, leveraging a hooking framework to inject data directly into Android's NFC daemon. **NFCMultiPay** takes the opposite approach: no native code, no root requirement. A pure-Java NFC reader that routes every exchange through a cloud broker. What unites them is not their architecture but **their origin**. DevilNFC's internal artifacts, log strings, C2 interface language, error messages, and social engineering copy are exclusively native **Spanish**. NFCMultiPay's

operator layer tells a consistent story: the panel interface, debug diagnostics, and relay core log strings are **Portuguese (Brazilian)** throughout.

Figure 1 - Notable NFC Relay Malware Families Observed Over the Past Year

Reading the two families together reveals a clear progression. Not two isolated discoveries, but two consecutive stages of the same **structural shift** in the threat landscape.

Through 2024 and into 2025, local TAs across the globe were primarily **consumers** of Chinese-developed NFC relay tooling: they bought access to MaaS platforms, ran campaigns against regional targets, and owned nothing of the underlying stack. The expertise, the infrastructure, and the development roadmap all remained in Chinese-speaking hands.

NFCMultiPay marks the first stage of a break from this model: a non-Chinese actor has built and deployed their own NFC relay toolkit independently. The separation is not yet clean. Deep inside the core relay functions, the original developer comments are written in **Simplified Chinese**. The evolution between the two identified builds makes the intent visible: in the second variant, those same Chinese strings were **replaced with functional English equivalents**. Every layer built around that core tells a different story: the C2 connection logic, error reporting, and debug diagnostics are commented and logged in Portuguese (Brazilian). This split points to a specific development pattern: the TA likely obtained a Chinese-authored relay component through a code leak and built their operational layer around it, leaving the core relay logic largely intact rather than reimplementing it from scratch. Cleafy has compared the Chinese-annotated functions against every NFC relay family currently tracked; none match, suggesting the upstream component has never been publicly documented.

DevilNFC represents the next stage. Its technical foundation is NFCGate, the same open-source starting point used by every prior family in this lineage, but everything built on top of that foundation is original: developer comments are written exclusively in **Spanish**, no Chinese artifacts appear anywhere in the stack, and no borrowed patterns survive from prior families. What NFCMultiPay achieved partially, DevilNFC completes: a TA that has fully internalized the relay primitive and now develops against it independently.

Figure 2 - Chinese log strings in NFCMultiPay replaced with English equivalents in the subsequent build

The timing is not coincidental. Both families carry indicators consistent with **AI-assisted development**. In DevilNFC, the phishing templates retrieved from the live C2 are **over-engineered** relative to their functional requirements: CSS and JavaScript structured with architectural precision, edge-case error handling, and a deliberate fake verification error on the first card tap that silently extends the relay window. NFCMultiPay's indicators differ but are equally revealing. The operator-facing debug diagnostics are structured log output with **emoji-categorized** metric labels separated by dense **ASCII box-drawing borders**. This formatting pattern is characteristic of LLM-generated logging scaffolding: a human developer building a diagnostic tool typically uses a logging framework. The implication is direct: building a

functional NFC relay toolkit no longer requires mastery of Android internals. It requires understanding what needs to be built, access to an open-source NFC relay framework such as NFCGate, an uncensored local LLM to scaffold the implementation without safety controls, and, optionally, a leaked malware codebase as a reference for the more complex components.

Figure 3 - AI artifacts in both malware families

This observation is **independently confirmed** by a recent public investigation by ESET Research, which exposed a new NGate variant targeting Brazilian users, in which the injected malicious code exhibits the same AI-assisted development pattern and Portuguese (Brazilian) logs. The variant trojanizes HandyPay, a legitimate NFC relay application, patching it with malicious PIN exfiltration code rather than building a new APK from scratch; a further indicator of how TAs in this cluster are **lowering their development overhead** by repurposing existing functional components rather than authoring original implementations. Taken together, the **three independently identified campaigns** converging on the same language, the same AI development indicators, and the same target geography suggest that a broader Portuguese (Brazilian) and Spanish-speaking TA cluster is actively entering the NFC relay space, with Europe and LATAM as their primary targets.

Technical Analysis

NFC relay is a well-documented technique, and its core mechanics have been covered extensively in prior public reporting, including Cleafy's analysis of SuperCard X. This section focuses exclusively on the features that distinguish DevilNFC and NFCMultiPay from every previously documented family.

Social Engineering and PIN Harvesting

Both DevilNFC and NFCMultiPay treat **PIN harvesting** as a core design requirement, not an afterthought: the victim is tricked into revealing their card PIN as an integral step of the attack flow, enabling chip-and-PIN transactions and ATM withdrawals that relay-only attacks cannot authorize.

DevilNFC

The compromise begins with a phishing message delivered via SMS or WhatsApp, directing the victim to a landing page impersonating the Google Play Store, presenting the malicious application as a mandatory security update from a legitimate Spanish banking institution. The

following screenshots were recovered directly from a victim's device during a **Cleafy Incident Response** engagement in March 2026, and represent the attack chain as it unfolded in the wild.

Figure 4 - Phishing messages & OTPs / Whatsapp Delivery / Malware Download

On launch, DevilNFC immediately locks the device using Android's **Kiosk Mode**, showing a **social engineering template** dynamically fetched by the C2. KioskActivity hides the system UI and overrides the hardware back button with an intentionally empty handler, **trapping the victim** inside the fake banking interface with no exit.

Figure 5 - Kiosk Mode for Card Reading and Pin Harvest

Once the victim is locked in, *SmsPermissionManager* silently **polls incoming SMS messages** in the background, intercepting any OTPs dispatched by the bank shortly after delivery and forwarding them to a dedicated C2 endpoint, which then routes them to the attacker's private Telegram channel in real time.

Figure 6 -Pin Harvest via Telegram and C2

The **PIN harvest** itself is triggered after the first card tap. A fake verification pop-up (pictured in the screenshot above) rendered by the remote template prompts the victim to enter their 4-digit card PIN. The value is immediately exfiltrated to two destinations simultaneously: the C2 endpoint *api_pin.php* and a hardcoded Telegram bot, transmitted in plaintext alongside the bank name and the victim's public IP address. The interface then deliberately triggers a fake verification error, instructing the victim to hold the card for an additional 10 seconds, a technique that extends the relay window to ensure the fraudulent transaction completes before the success screen is displayed.

NFCMultiPay

NFCMultiPay's social engineering approach is simpler but equally effective. The victim receives an APK impersonating the target bank's official application and is walked through installation outside the Play Store. Unlike DevilNFC, where the relay session is established transparently in the background, NFCMultiPay requires the fraudster to maintain active contact with the victim throughout the operation: the session is initiated by a **connection PIN** supplied out-of-band, typically via phone call or messaging platform, meaning the victim must be guided to open the app, enter the PIN, and then present their card.

Figure 7 - NFCMultiPay Pin Exfiltration View

Once connected, the relay flow proceeds through a **guided UI sequence**: the victim is instructed to hold their card against the device for reading, then prompted to enter their card

PIN before the relay completes. The PIN is published unencrypted to the MQTT topic `nfc/relay/<pin>/card_ready`, alongside the last 4 PAN digits and the card brand.

The Dual-Role APK in DevilNFC

DevilNFC's most significant technical contribution to the NFC relay landscape is its asymmetric execution model, which Cleafy refers to as a “**Dual-Role APK**” design:

- On the **victim's device**, which is typically not rooted, system-level hooks silently fail to initialize, and the application falls back to the **passive reader** role, using only standard Android APIs with no suspicious behavior visible to static or dynamic scanners.
- On the **attacker's rooted device**, the same APK transforms into a real **host card emulator**.

Figure 8 - Attacker's Mode on Rooted Device

The transformation is driven by the **Xposed Framework**, which injects DevilNFC's hooking module directly into `com.android.nfc`, the Android system process responsible for all NFC communications on the device. The native engine behind this is `libnfcgate.so`, a direct derivative of the NFCGate open-source research toolkit, loaded dynamically into the NFC daemon process at runtime.

The Host Card Emulation Process

To emulate a card through Android's standard **Host Card Emulation** (HCE) framework, the tapper-side application must declare which payment **Application Identifiers** (AIDs) it handles in a static XML resource file, referenced from the manifest via the `android.nfc.cardemulation.host_apdu_service` metadata entry. This declaration is what allows Android to route incoming **Application Protocol Data Unit** (APDU) traffic from a POS terminal to the correct application.

Here below, we will describe how each family has handled this case:

- **SuperCard X**: represents the earliest and most detectable approach. A comprehensive list of real financial AIDs, declared explicitly in the `hce.xml` (Visa, Mastercard, etc.), making the application's intent immediately visible to any static analysis engine.

- **NFU Pay**: registers only the **Proximity Payment System Environment** (PPSE) identifier *2PAY.SYS.DDF01 (325041592E5359532E4444463031)*, the directory service a POS terminal always queries first to discover which payment AIDs a card supports. By responding to that opening command, NFU Pay's HostApuService becomes the **active handler for the entire NFC session**: Android's HCE framework routes all subsequent APDUs from that session to whichever service won the initial AID match, regardless of which payment AID the terminal selects next.
- **DevilNFC**: declares in its *hce.xml* a single non-payment identifier (*F0010203040506*) lifted directly from Google's official Android HCE documentation sample code. Under normal conditions, when a POS terminal sends a *SELECT AID* for a real payment application such as Visa or Mastercard, Android resolves the request via its standard AID routing table and forwards traffic to the registered handler that claims that AID. DevilNFC claims none of them. The Xposed hook on *HostEmulationManager.findSelectAid()* solves this by intercepting the internal AID lookup before Android resolves it, remapping any incoming payment AID to DevilNFC's registered dummy identifier and forcing Android to route all subsequent APDU traffic into DevilNFC's relay pipeline, regardless of what the terminal originally requested. *F0010203040506* never leaves the attacker's device: it is purely an internal routing handle, while the actual transaction is handled by *libnfcgate.so* relaying live APDUs against the victim's physical card.

Figure 9 - Difference between Host Card AIDs Declaration Approaches

NFCMultiPay cannot be placed in this progression with confidence, as the **tapper-side component has not yet been recovered**. What can be observed is that the reader-side sample is notably cleaner than its counterparts: the reader components of SuperCard X and NFU Pay both carry HCE declarations in their manifests, despite not requiring them for card reading, whereas NFCMultiPay's reader declares only *android.permission.NFC*, which is the bare minimum the Android framework requires for ISO-DEP tag reading.

C2 Protocol and Network Communications

DevilNFC

The **APDU relay runs on a persistent raw TCP socket and uses** Google Protocol Buffers for binary serialization. A four-opcode state machine *OP_SYN*, *OP_ACK*, *OP_PSH*, *OP_FIN* handles session pairing and bidirectional frame routing, with *TCP_NODELAY* set explicitly to minimize latency against the POS terminal's response timeout. Once the victim holds the card against the device, every command from the POS travels through this tunnel to the victim's physical card, and every response travels back in milliseconds, within the window the terminal expects.

After the relay is established, the WebView template displays a countdown and then a four-digit PIN entry pop-up (as pictured in the **Figure 5**). The PIN is exfiltrated simultaneously to a dedicated C2 endpoint and to the attacker's Telegram channel, in plaintext, alongside the target institution name and the victim's public IP address. A second form captures the online banking password through the same channel. A deliberately triggered fake error on the first card tap forces the victim to hold the card again for ten seconds, not a bug, but a designed extension of the relay window, ensuring the transaction completes before success is displayed.

Figure 10 - DevilNFC C2 Panel

Configuration parameters extracted from the malware and traffic observed during dynamic analysis enabled us to identify the active C2 infrastructure operated by the threat actors behind the DevilNFC Campaign, as shown in the screenshot above.

NFCMultiPay

NFCMultiPay's two identified variants implement the same logical relay protocol but differ significantly at the transport layer, providing a clear view of how the toolkit evolved over versions.

The first variant relies on an **HTTPS REST polling model**. The victim-side app validates the session PIN against `/api/nfc/check-pin`, then enters a continuous polling loop on `/api/nfc/poll/<pin>` waiting for inbound APDU commands from the tapper. Responses are published back via POST to `/api/nfc/publish`. This variant also contains a fully stubbed MQTT client that is never invoked: its topic namespace is defined, but the connection method is empty, indicating that the developer had already planned the transport upgrade but did not complete it.

The second variant completes that transition. REST polling is replaced entirely by **event-driven MQTT** over TCP port 1883, eliminating the polling latency that characterized the first version. The broker hostname is no longer stored in cleartext but encrypted via a multi-stage AES/CBC and XOR scheme. The session model is otherwise identical: the victim-side app subscribes to `nfc/relay/<pin>/apdu_request` for inbound commands and publishes responses to `nfc/relay/<pin>/apdu_response`, with the tapper operating on the other end of the same broker channel.

Figure 11 - NFCMultiPay PIN Exfiltration

One operationally significant detail introduced in the MQTT variant is the use of **retained messages** on the *nfc/relay/<pin>/card_ready* topic, which carry the victim's card PIN, the last 4 PAN digits, and the card brand. Published with *retain=true*, the payload persists on the broker indefinitely, so any tapper that connects or reconnects after the victim's PIN entry immediately receives the stored credentials without further interaction with the victim.

Beyond the relay infrastructure, the same IP address exposed an operator-facing web panel, written entirely in Portuguese (Brazilian), through which the fraudster manages active relay sessions, providing a direct view into the operational layer above the malware itself.

Figure 12 - NFCMultiPay C2 Panel

Conclusions

DevilNFC and NFCMultiPay represent a concrete shift in the landscape of NFC relay threats, but their implications extend well beyond this specific malware category. What these two families demonstrate is a **change in the economics of malware development**: local groups of fraudsters, who previously had no choice but to purchase access to Chinese-developed MaaS platforms, can now build their own tooling independently.

NFC relay malware is, by design, a relatively constrained category. The functional requirements are narrow: read a card, forward the APDU stream, and collect a PIN. There is no persistence engine to build, no evasion framework to maintain, no complex C2 protocol to design from scratch. The **open-source foundation** (e.g., NFCGate) has been publicly available for years. These constraints make NFC relay an **ideal entry point** for TAs using AI-assisted development tools: the scope is small enough that an LLM can scaffold a working implementation from a clear specification, and the open-source foundation removes the hardest problems entirely. The same dynamic will not remain confined to NFC relay: Android RATs and banking trojans are more complex, but the gap is narrowing. Two parallel trends are accelerating it. First, **open-source LLM models** can be run locally **without any of the safety controls** enforced by commercial providers, making them directly usable for malware development without content filtering. Second, **leaked malware source code** accumulating in public repositories plays the same role as NFCGate did for NFC relay: a working foundation that removes the hardest implementation problems and reduces the task to integration and customization rather than original development.

Appendix - Indicators of Compromise (IOCs)

Type	Value	Description
Domain	nfc Crackatm[.]com	DevilNFC C2 / Relay Server
Domain	spicyNagets[.]shop	DevilNFC C2 / Relay Server
IPV4	185.203.116[.]18	NFCMultiPay C2
IPV4	47.253.167[.]219	NFCMultiPay C2
MD5	caa5e8cf3275339d251210072ebe88c2	DevilNFC Apk Sample
MD5	35dd9c3a56e88a39bf6c8fdad46b0398	NFCMultiPay Apk Sample
MD5	9d19527aeb4cabfb40bbaea6d73b5ff0	NFCMultiPay Apk Sample
Package Name	com.devilnfc.reader	DevilNFC APK Package Name